Lesson 06 · VB.NET for Legacy Estates

Read, run and retire Visual Basic .NET — the syntax map, the semantics that change numbers, VB ↔ C# interop and a parity-tested migration

Contents

1 · Why Visual Basic is your problem, not your language 1
- 1.1 · The numbers behind this lesson 2

2 · Where VB.NET lives — and what .NET 10 still runs 2
- 2.1 · Visual Basic on .NET Framework and on .NET 10 2
- 2.2 · Where each VB.NET workload goes on .NET 10 3
- 2.3 · VB templates shipped with the pinned SDK 3
- 2.4 · Visual Basic .NET, release by release 4

3 · Reading VB fluently — a map from the C# and Java you already read 4
- 3.1 · C# and Java → Visual Basic 4
- 3.2 · Six VB constructs with no direct C# spelling 5
- 3.3 · The declaring end — controls declared WithEvents in the designer file 6
- 3.4 · The handling end — most of your VB reading skills in one method 6

4 · Semantics that bite — where a line-by-line port changes numbers 6
- 4.1 · The top of a legacy file — two option lines and an implicit conversion 6
- 4.2 · Loose on purpose — a file that opts out of Option Strict 7
- 4.3 · On Error Resume Next — the error that never happened 7
- 4.4 · And, AndAlso, IIf and If() — which right-hand sides run 7
- 4.5 · Upper bounds, the two divisions and checked arithmetic 7
- 4.6 · What the VB console prints on .NET 10 8
- 4.7 · Four midpoints, three rounding rules 8
- 4.8 · Java instincts vs the VB semantics a faithful port must keep 8

5 · VB ↔ C# interop — one runtime, two compilers' rules 8
- 5.1 · C# calling a VB Module — the adapter at the seam 9
- 5.2 · A partner's C# SDK — valid C#, awkward VB 9
- 5.3 · The VB caller — copy-back through ByRef, and the escape hatch 9
- 5.4 · Two compile errors worth recognising 10
- 5.5 · Interop rules at the boundary 10

6 · Migration strategy — characterize, strangle, retarget, port 10
- 6.1 · Characterization tests in VB — pinning the quirks on purpose 10
- 6.2 · Choosing a route for one VB project 11
- 6.3 · The life of one project in the migration 11
- 6.4 · One quote through the strangler — VB screen, C# rule, VB oracle 12
- 6.5 · Migration tooling this lesson owns, checked in September 2026 12
- 6.6 · Convert with a tool, or rewrite by hand? 12

7 · Parity testing — prove the port before anyone switches 12
- 7.1 · The same loadings in the VB original and the faithful C# port 13
- 7.2 · The discount table and the total — VB original and C# port 13
- 7.3 · The injected changes — one enum member, one line of the port 13
- 7.4 · The grid — boundary values, multiplied out 13
- 7.5 · The parity gate in xUnit 14
- 7.6 · Checking VbCompat against the Visual Basic runtime 14
- 7.7 · Premiums changed by one port change 15
- 7.8 · Values per grid dimension 15

8 · Hands-on — run the estate in miniature 15
- 8.1 · Commands 15
- 8.2 · What you should see 16
- 8.3 · Lines of code by migration role 16
- 8.4 · Sample code by language 16

Page numbers are resolved from the rendered PDF. VB and C# panels are cut from the compiled samples; numbers marked measured come from real runs of those samples.

1 · Why Visual Basic is your problem, not your language

You will rarely write new Visual Basic, but you will own systems written in it. Line-of-business estates built in the 2000s — quote engines, back-office Windows Forms tools, Web Forms portals — carry VB.NET beside C#. In a migration inventory they look exactly like the Java EE and JSP applications you have already maintained: the authors are gone, business rules sit inside event handlers, and there are no tests. The job is the same too: read it, keep it running, and plan its exit without changing a single premium.

The language is stable by policy, so what you learn here will not go stale. Microsoft's stated strategy is that Visual Basic keeps a stable design, adopts new runtime features *consumption-only*, will not be extended to new workloads, and keeps investing in core scenarios such as Windows Forms and libraries (Visual Basic language strategy). The current version, Visual Basic 17.13, ships with Visual Studio 2026 and adds recognition of two runtime features rather than new syntax (What's new for Visual Basic).

The risk is not syntax, it is semantics. VB reads like pseudocode, so a port to C# looks like typing. It is not: implicit conversions round half to even, `/` on integers returns a Double, `And` evaluates both sides, `DateDiff` subtracts year numbers, and string comparison can ignore case file by file. This lesson measures what each quirk does to real quotes: a line-by-line port that gets just one of them wrong changes between 6.1% and 44.9% of 50,176 premiums.

It ends with a method, not a converter. VB characterization tests (the golden-master technique) pin today's behaviour, a faithful C# port reproduces it on purpose, and a parity grid proves the two agree before traffic moves. They are the mirror image of the specification tests behind a 100%-coverage standard: those say what the code should do, these say what it does.

1.1 · The numbers behind this lesson

VISUAL BASIC	VB TEMPLATES	PARITY GRID	WORST PORT MISTAKE	LESSON 06 SAMPLES
17.13	**14**	**50,176**	**44.9%**	**655 lines**
current version · Visual Studio 2026 · verified	SDK 10.0.401 · none for web or worker · measured	quote requests · measured	premiums changed by (int) casts · measured	VB 291 · C# 364 · 9 projects · measured

KEY: • Same idea • Renamed • Different • Trap • Verified • Estimate • Measured VB C# legacy architecture tooling

i Before you start

1. You need lesson 01 (The .NET Platform Map) (the two .NETs, target frameworks, VB as a consumer of C# types) and lesson 02 (C# Language Essentials) (`decimal`, nullability, exceptions). VB's `Async/Await` is covered in lesson 05 (Async, Tasks & Concurrency).
2. You will build nine projects: a legacy-style VB rating library with VB xUnit characterization tests, a faithful C# port, a parity kit with its xUnit tests and a console report, a C# partner SDK consumed by a VB console that demonstrates the traps, and a VB Windows Forms form that calls the C# port.
3. Premiums use the MotorQuote running example on the track's one canonical tariff (curriculum, "Canonical tariff"): every rate, loading and discount is **illustrative, not a real tariff**. One deliberate variant: this lesson's legacy VB prices in whole baht with banker's rounding, so its worked example is 8,933 where the canonical decimal tariff gives 8,933.71 (§8.2).
4. `L06.WinFormsVb` targets `net10.0-windows` and is build-only; every other sample targets plain `net10.0` with no Windows-only API (verified on Windows only).

2 · Where VB.NET lives — and what .NET 10 still runs

Most VB.NET you meet runs on .NET Framework, and its workload — not its language — decides its future. Class libraries, console jobs and Windows Forms tools have a direct path to modern .NET. ASP.NET Web Forms has none: it is a .NET Framework technology, and Microsoft's migration guidance moves those pages to Blazor on ASP.NET Core (Migrate from ASP.NET Web Forms to Blazor). The porting map for `System.Web`, WCF and IIS belongs to lesson 12 (Cloud-Native .NET on AWS & Modernization).

On .NET 10, VB is first-class for libraries, console apps, tests and Windows desktop — and has no web or worker template. The 2020 plan for .NET 5 listed class library, console, Windows Forms, WPF, worker service and ASP.NET Core Web API (Visual Basic support planned for .NET 5.0), and the Windows Forms Application Framework arrived with .NET 5 (Visual Basic WinForms apps in .NET 5). The SDK this track pins ships 14 VB templates — libraries, console, three test frameworks, Windows Forms and WPF — and none for web or worker projects (measured with `dotnet new list --language VB`). This lesson builds no VB web project, so read the two zero bars as "no template, not demonstrated here", not as "impossible". Razor pages and Blazor components are C#: Razor syntax "consists of Razor markup, C#, and HTML" (Razor syntax reference). The practical shape is a C# host calling VB libraries, which lesson 08 (ASP.NET Core Web APIs) builds.

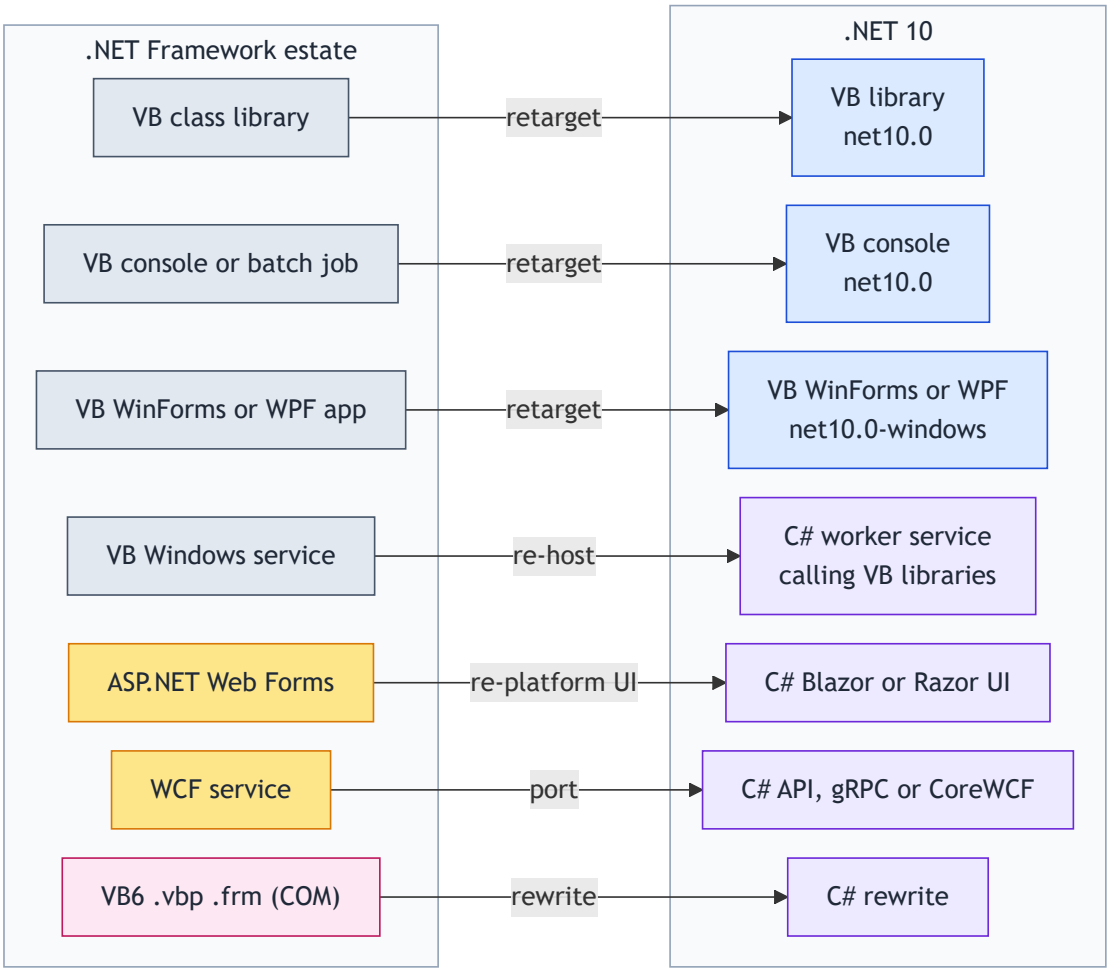
Visual Basic 6 is a different animal. A `.vbp/.frm/.bas` project is COM-era VB6, not .NET. Its runtime is supported for the lifetime of supported Windows versions, but the VB6 IDE has been unsupported since 8 April 2008 (Support Statement for Visual Basic 6.0). There is no retarget, only a rewrite.

So triage by workload first, language second. A VB class library on `net48` is a retarget and a test run; a Web Forms page is a new UI, whatever language its code-behind uses.

2.1 · Visual Basic on .NET Framework and on .NET 10

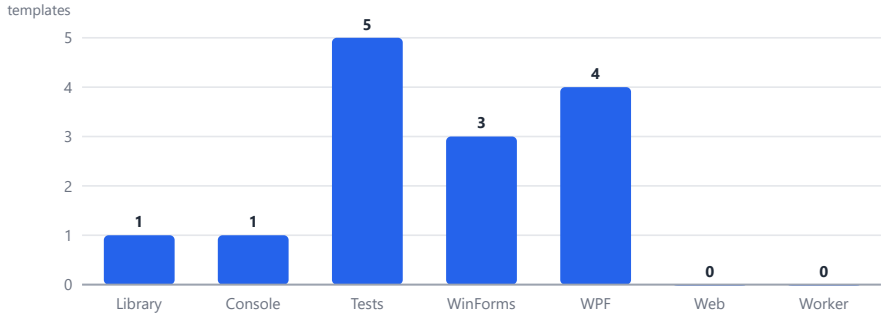
Workload	VB on .NET 10	VB template in SDK 10.0.401	Route
Class library	• Supported	`classlib`	Retarget to `net10.0`; port to C# when the code changes
Console app	• Supported	`console`	Retarget
Unit tests	• Supported	`xunit` · `nunit` · `mstest` + 2 item templates	Keep VB tests as the characterization suite until the port lands
Windows Forms	• Supported · Windows	`winforms` · `winformslib` · `winformscontrollib`	Retarget to `net10.0-windows`; move rules out of handlers
WPF	• Supported · Windows	`wpf` + 3 library templates	Retarget; port view-models to C# first
Worker / Windows service	• Planned 2020 · no template	—	Not demonstrated here — host in a C# worker, see lesson 08 (ASP.NET Core Web APIs); keep VB libraries behind it
ASP.NET Core Web API	• Planned 2020 · no template	—	Not demonstrated here — C# host, VB libraries referenced as projects
Web Forms · Razor · Blazor	• Not VB	—	Web Forms is Framework-only; Razor and Blazor are C# — re-platform the UI
Visual Basic 6	• Not .NET	—	Rewrite from a behaviour specification

2.2 · Where each VB.NET workload goes on .NET 10



grey = Framework workload with a direct path · amber = web-era Framework workload · rose = not .NET at all · blue = stays VB on .NET 10 · violet = moves to C#

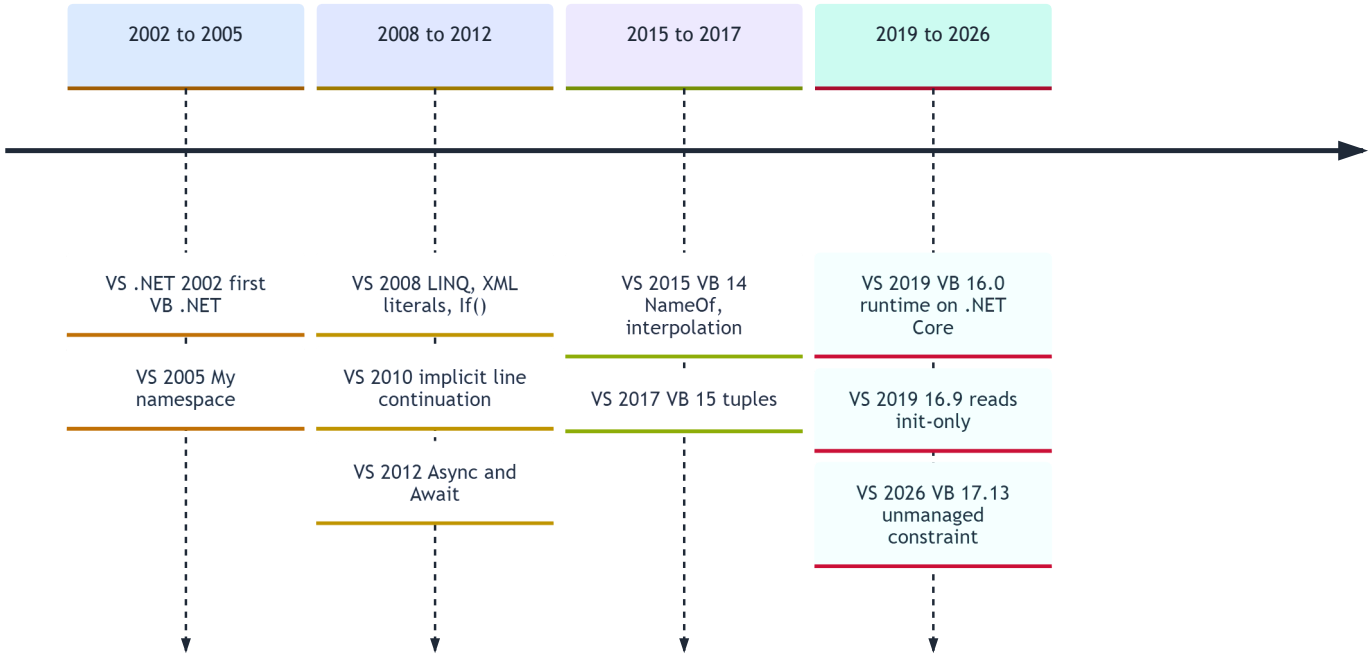
2.3 · VB templates shipped with the pinned SDK



measured: dotnet new list --language VB on the build machine, SDK 10.0.401, grouped by workload · Tests = 3 project + 2 item templates

> **The two zero bars are the policy made visible.** Web API and worker projects were on the 2020 plan yet have no VB template, and the strategy rules out new workloads such as web front ends. When a plan says "modernise the VB web app", read it as "write a C# host".

2.4 · Visual Basic .NET, release by release



date a legacy file by the syntax it uses: If() means Visual Studio 2008 or later, $"..." and NameOf 2015 or later, tuples 2017 or later · from Microsoft's What's new for Visual Basic

3 · Reading VB fluently — a map from the C# and Java you already read

VB is C#'s semantics spelled in English keywords — until it is not. Both compile to the same IL and share types, generics, exceptions, async and LINQ, so most of a VB file maps word for word: `Dim x As Integer` is `int x`, `Shared` is `static`, `Friend` is `internal`, `MustInherit` is `abstract`. Read the table once; the rows marked • Trap are the only real obstacles.

It is line-oriented and case-insensitive. A statement ends at the line break unless the line ends with a comma, an operator or an open parenthesis (implicit continuation, Visual Studio 2010) or with an explicit _. `premium` and `Premium` are the same identifier, which starts to matter the moment VB calls a C# library (§5).

Four file options change meaning, not style. `Option Explicit`, `Option Strict`, `Option Infer` and `Option Compare` sit at the top of a file and override the project. Visual Studio's initial default for Option Strict is Off (Option Strict statement); this repository's `Directory.Build.props` turns it On. Always read the first lines of a legacy file before trusting what = or / does below them.

Events are declared, not subscribed. A form declares its controls `WithEvents`, and a handler method ends with `Handles QuoteButton.Click`. There is no += anywhere, so finding "who calls this" means searching for `Handles` clauses.

3.1 · C# and Java → Visual Basic

C# / Java	Visual Basic	Kind	Watch for
`int x = 0;`	`Dim x As Integer = 0`	• Renamed	`Dim x = 0` infers the type when Option Infer is On
`null`	`Nothing`	• Trap	Nothing assigned to an Integer gives 0, and `"" = Nothing` is True (Nothing)
== on references	`Is` / `IsNot`	• Renamed	Use `Is Nothing`, never = `Nothing`. A type that overloads = (`String`, a C# record) compares by value with =, while `Is` compares identity; on a plain class = does not compile (BC30452) — lesson 03 (Types, OOP & Generics)
&& / ||	`AndAlso` / `OrElse`	• Renamed	Only these short-circuit
& / | on booleans	`And` / `Or`	• Trap	Legacy code uses them for logic; both sides always run (And)
`c ? a : b`	`If(c, a, b)`	• Same idea	`IIf(c, a, b)` is a function: both arms run and it returns Object
/ on integers	`\`	• Trap	VB / is floating-point division: integral operands give Double, Decimal stays Decimal (/ operator)
%	`Mod`	• Renamed	Same sign rules as C#
+ for strings	&	• Trap	With Option Strict Off, `"42" + 1` is 43 and `"42" & 1` is "421"
`static`	`Shared` · `Module`	• Renamed	A Module's members are callable unqualified from anywhere in the project
`internal` · `protected internal`	`Friend` · `Protected Friend`	• Renamed	—
`abstract` · `sealed` · `virtual`	`MustInherit` · `NotInheritable` · `Overridable`	• Renamed	Methods are non-virtual by default in both — lesson 03 (Types, OOP & Generics)
`new` (hiding)	`Shadows`	• Different	Shadows hides by name, every overload

C# / Java	Visual Basic	Kind	Watch for
`this` · `base`	`Me` · `MyBase` · `MyClass`	• Different	`MyClass` calls this class's version even when overridden
`(int)x` · `x as T`	`CInt(x)` · `TryCast` · `DirectCast` · `CType`	• Trap	`CInt` rounds half to even; the C# cast truncates (CInt)
`new decimal[6]`	`Dim ncb(5) As Decimal`	• Trap	The number is the upper bound: six elements (arrays)
`Array.Resize`	`ReDim Preserve`	• Renamed	Allocates a new array every time
`button.Click += OnClick`	`WithEvents` + `Handles` · `AddHandler`	• Different	Wiring is declared on the field and the method
`ref` · `out`	`ByRef`	• Trap	A property passed ByRef is written back; the default is ByVal (ByRef)
`catch (E e) when (…)`	`Catch e As E When …`	• Same idea	`On Error GoTo` and `Resume Next` still compile (On Error)
`using System;`	`Imports System`	• Renamed	Projects add further imports in the `.vbproj`
`CompareInfo.Compare(a, b, IgnoreCase | …)`	`Option Compare Text` + `=`	• Trap	One file-level line changes every `=` and `Select Case` on strings. `OrdinalIgnoreCase` is not the same rule: culture, width and kana differ (§4.8)
`new Foo { X = 1 }`	`New Foo With {.X = 1}` · `With obj … End With`	• Different	Inside a With block a leading `.` binds to the block's object; nested blocks hide which object a line writes
`void M()` · `T M()`	`Sub` · `Function … As T`	• Renamed	`Return x` works in both; legacy code assigns the value to the function's own name

3.2 · Six VB constructs with no direct C# spelling

reading view

1 Module VB • Different

A type whose members are all `Shared` and visible to the whole project unqualified.

C#	`static class` plus `using static`
TRAP	A bare `CalcPremium(…)` call hides which file owns the rule
PORT	A static class — C# already calls a Module's members as static members

2 WithEvents and Handles VB • Different

Event wiring declared on the field and on the handler method.

C#	`+=` in the constructor or `InitializeComponent`
TRAP	Handlers look unreferenced; search for `Handles X.Click`
PORT	Regenerate designer wiring, then review every handler for business rules

3.2 · Six VB constructs (continued)

reading view

3 The My namespace VB legacy • Different

Generated shortcuts to the application, computer, settings and resources, per project type.

C#	`Environment`, `Application`, configuration — no single equivalent
ON .NET	A VB console on .NET 10 has no `My.Computer`: BC30456 (§5.4)
PORT	Replace call by call; `My.Settings` becomes configuration — lesson 08 (ASP.NET Core Web APIs)

4 On Error GoTo and Resume Next VB legacy • Trap

Unstructured error handling: jump to a label, or ignore the error and carry on.

C#	None; Resume Next behaves like an empty catch around every statement
MEASURED	Err.Number 11 is set, execution continues, the result stays 0
PORT	Characterize first — the swallowed errors are behaviour callers rely on

3.2 · Six VB constructs (continued)

reading view

5 ByRef that accepts a property VB C# • Trap

VB reads the property, passes a temporary, and writes the result back.

C#	`ref` never accepts a property (a compile error)
MEASURED	`AddFee(line.Total)` changes it; `AddFee((line.Total))` does not
PORT	Spell it out: copy to a local, pass `ref`, assign back

6 Option Compare Text VB • Trap

A file-level switch: string `=` and `Select Case` ignore case, using the current culture (Option Compare statement).

C#	A `StringComparison` or `CompareOptions` on every call
MEASURED	`"CLASS1" = "class1"` is False in one file and True in the next
PORT	One named helper, `VbCompat.TextEquals`, not scattered `ToUpper()`

3.3 · The declaring end — controls declared WithEvents in the designer file

Visual Basic — samples/L06.WinFormsVb/QuoteForm.Designer.vb · #region withevents

```
Friend WithEvents CoverCodeBox As TextBox
Friend WithEvents SumInsuredBox As NumericUpDown
Friend WithEvents BirthDatePicker As DateTimePicker
Friend WithEvents QuoteButton As Button
Friend WithEvents ResultLabel As Label
```

This is where the events are wired, and no += appears. A field declared `Friend WithEvents` can be the target of a `Handles` clause anywhere in the class. The form designer owns and rewrites this file, so business logic never belongs in it.

3.4 · The handling end — most of your VB reading skills in one method

Visual Basic — samples/L06.WinFormsVb/QuoteForm.vb · #region handles-click

```
' Handles wires the event of a control declared WithEvents in the
' designer file. The rating now lives in the C# port; the VB form
' only gathers input and shows the result (strangler step 1).
Private Sub QuoteButton_Click(sender As Object, e As EventArgs) _
        Handles QuoteButton.Click
    Dim request As New QuoteRequest(
        CoverCodeBox.Text, SumInsuredBox.Value,
        DateOnly.FromDateTime(BirthDatePicker.Value),
        DateOnly.FromDateTime(Date.Today),
        LicenceMonths:=120, Claims:=0)

    Dim premium As Decimal? = PremiumCalculator.Calculate(request)
    ResultLabel.Text = If(premium.HasValue,
                          $"{premium.Value:N0} THB (illustrative)",
                          "Declined")
End Sub
```

Read it line by line. `Handles QuoteButton.Click` subscribes to the control declared above; the trailing _ continues the declaration. Sub is a method that returns nothing (`Function … As T` returns a value). `New QuoteRequest(…)` constructs a C# record, mixing positional and named arguments (`LicenceMonths:=120`). `Decimal?` is the C# `decimal?` the port returns, and `If(…)` is the ternary. The form stays VB; the rating rule has already moved to C#.

4 · Semantics that bite — where a line-by-line port changes numbers

Option Strict Off is where most legacy surprises live. With it off, VB converts silently between strings, numbers and dates and resolves members at run time (late binding) (Option Strict statement). The rating module in this lesson opens with `Option Strict Off` and `Option Compare Text` (panel 4.1): two lines that decide how every expression below them behaves. A file option overrides the project, so a solution-wide Strict On does not make every file strict — the sample `Loose.vb` compiles loosely inside a strict project.

Conversions to integers round half to even. `CInt`, `CLng` and every implicit Double-to-Integer conversion round a fraction of exactly .5 to the nearest even integer: 2.5 becomes 2 and 3.5 becomes 4 (Type conversion functions). A C# `(int)` cast truncates, while `Math.Round` defaults to the same round-half-to-even rule (Math.Round). The Java instinct, round half up, is wrong for both.

/ is floating-point division, not integer division. In VB, `18 / 12` is 1.5 for any two integral operands (/ operator); assign it to an Integer and banker's rounding stores 2, so an 18-month licence counts as two years. Integer division is \. A C# port that writes `months / 12` gets 1.

Five more quirks change results. `DateDiff(DateInterval.Year, …)` uses only the year parts, so a driver born on 31 December is a year older on 1 January (DateAndTime.DateDiff). `Dim ncb(5)` declares an upper bound, giving six slots (Arrays in Visual Basic). Option Compare Text makes string = ignore case (Option Compare statement). And/Or evaluate both operands (And operator) — harmless in arithmetic, fatal when the right side dereferences something the left side just checked. And integer arithmetic is *checked*: unless a project passes `-removeintchecks` (-removeintchecks), `Integer.MaxValue + 1` throws `OverflowException`, where C# wraps silently unless the code says `checked`. It is the one VB default stricter than C#, and lesson 02 (C# Language Essentials) §8.3 measures both sides. A port written without `checked` drops a guard nobody wrote down, which is why `VbCompat.CInt` in panel 4.8 wraps its conversions in `checked(…)`.

4.1 · The top of a legacy file — two option lines and an implicit conversion

Visual Basic — samples/L06.LegacyRating/LegacyPremium.vb · top of file

```
Option Strict Off
Option Compare Text

' MotorQuote rating written the way many 2000s VB.NET estates were:
' loose file options, Double rates, a magic -1 for "declined" and
' Microsoft.VisualBasic helpers. The tariff VALUES are the track's
' canonical illustrative tariff (curriculum, "Canonical tariff");
' the whole-baht Integer rounding is this legacy code's own habit,
' kept on purpose - it is what the port must reproduce.
Public Module LegacyPremium

    Public Function CalcPremium(ByVal coverCode As String,
            ByVal sumInsured As Decimal, ByVal dateOfBirth As Date,
            ByVal startDate As Date, ByVal licenceMonths As Integer,
            ByVal claims As Integer,
            Optional ByVal commercial As Boolean = False,
            Optional ByVal engineCc As Integer = 1500,
            Optional ByRef declineReason As String = "") As Decimal
        Dim rate As Double
        Select Case coverCode.Trim()    ' Option Compare Text: "class1" matches
            Case "CLASS1" : rate = 0.021
            Case "CLASS2PLUS" : rate = 0.012
            Case "CLASS3PLUS" : rate = 0.009
```

```
            Case "CLASS3" : rate = 0.004
            Case Else
                declineReason = "unknown cover " & coverCode
                Return -1
        End Select
        Dim base As Integer = sumInsured * rate    ' Double -> Integer rounds
```

Read the first two lines before anything else. `Option Strict Off` lets `Dim base As Integer = sumInsured * rate` compile: a Decimal times a Double, stored in an Integer with no cast in sight. `Option Compare Text` is why `Case "CLASS1"` also matches `"class1"`. The `-1` after `Return` is the "declined" sentinel that §5.1 wraps in an adapter; §7.1 and §7.2 put the rest of this function beside its C# port.

4.2 · Loose on purpose — a file that opts out of Option Strict

Visual Basic — samples/L06.VbSemantics/Loose.vb · #region late-binding

```
Dim card As Object = New L06.PartnerSdk.PartnerRateCard()
Console.WriteLine($"late bound LoadingFor(2)  {card.LoadingFor(2)}")
Try
    Console.WriteLine(card.Discount)         ' compiles, fails at run time
Catch ex As MissingMemberException
    Console.WriteLine($"late bound Discount       {ex.GetType().Name}")
End Try
Console.WriteLine($"""42"" + 1  = {"42" + 1}      ""42"" & 1  = {"42" & 1}")
```

This compiles inside a project whose default is Option Strict On. `card` is an `Object`, so `LoadingFor(2)` is bound at run time and a misspelt member (`Discount`) fails only when that line runs, with `MissingMemberException`. `+` on a string and a number converts the string; `&` concatenates.

4.3 · On Error Resume Next — the error that never happened

Visual Basic — samples/L06.VbSemantics/Loose.vb · #region on-error

```
Dim zero = 0
On Error Resume Next
Dim ratio = 1 \ zero                         ' raises, execution continues
Console.WriteLine($"On Error Resume Next   Err.Number {Err.Number}, ratio {ratio}")
On Error GoTo 0
```

The division by zero raised, and nothing stopped. `Resume Next` continues at the next statement after every error, so `ratio` keeps its default 0 and only `Err.Number` (11, division by zero) records that anything went wrong; `On Error GoTo 0` switches the handling off again. Replace it with `Try/Catch` only after a test has pinned the 0.

4.4 · And, AndAlso, IIf and If() — which right-hand sides run

Visual Basic — samples/L06.VbSemantics/Program.vb · #region and-andalso

```
Dim hasPolicy = False
calls = 0
If hasPolicy And Touch() Then Console.WriteLine("unreachable")
Console.WriteLine($"And        right side ran {calls}x")
calls = 0
If hasPolicy AndAlso Touch() Then Console.WriteLine("unreachable")
Console.WriteLine($"AndAlso    right side ran {calls}x")
calls = 0
Dim label = IIf(hasPolicy, Touch(), "none")    ' a function: both run
Console.WriteLine($"IIf        right side ran {calls}x -> {label}")
calls = 0
Dim label2 = If(hasPolicy, Touch().ToString(), "none")
Console.WriteLine($"If()       right side ran {calls}x -> {label2}")
```

Count the calls, not the result. All four print the same answer, but `And` and `IIf` run `Touch()` anyway. In legacy code the right-hand side is often `policy.Holder.Age > 25` after a null check — with `And` that throws.

4.5 · Upper bounds, the two divisions and checked arithmetic

Visual Basic — samples/L06.VbSemantics/Program.vb · #region arrays-division

```
Dim ncb(5) As Decimal                ' upper bound 5, not length 5
Console.WriteLine($"Dim ncb(5)          Length {ncb.Length}")
ReDim Preserve ncb(7)
Console.WriteLine($"ReDim Preserve (7)  Length {ncb.Length}")

Dim months = 18
Console.WriteLine($"18 / 12 = {months / 12}   18 \ 12 = {months \ 12}")
Console.WriteLine($"18 Mod 12 = {months Mod 12}")
Console.WriteLine($"CInt(18 / 12) = {CInt(months / 12)}   CInt(11 / 12) = {CInt(11 / 12)}")

Dim big As Integer = Integer.MaxValue          ' checked unless -removeintchecks
Try
    Console.WriteLine($"Integer.MaxValue + 1  {big + 1}")
Catch ex As OverflowException
    Console.WriteLine($"Integer.MaxValue + 1  {ex.GetType().Name}")
End Try
```

`Dim ncb(5)` has Length 6. `ReDim Preserve ncb(7)` copies into a new array of 8. `/` gives 1.5 and `CInt` of it gives 2; `\` gives 1. `CInt(11 / 12)` is 1 — VB rounds to nearest, it never truncates. `Integer.MaxValue + 1` throws where the same C# line would wrap to `-2147483648`.

4.6 · What the VB console prints on .NET 10

Output — L06.VbSemantics (1/2) captured on the build machine

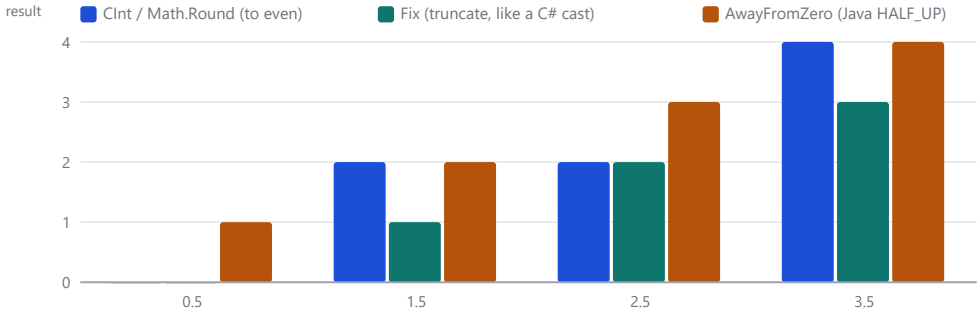
```
And        right side ran 1x
AndAlso    right side ran 0x
IIf        right side ran 1x -> none
If()       right side ran 0x -> none
Dim ncb(5)          Length 6
ReDim Preserve (7)  Length 8
18 / 12 = 1.5   18 \ 12 = 1
18 Mod 12 = 6
CInt(18 / 12) = 2   CInt(11 / 12) = 1
Integer.MaxValue + 1  OverflowException
0.5  CInt 0  Fix 0  Math.Round 0  AwayFromZero 1
1.5  CInt 2  Fix 1  Math.Round 2  AwayFromZero 2
2.5  CInt 2  Fix 2  Math.Round 2  AwayFromZero 3
3.5  CInt 4  Fix 3  Math.Round 4  AwayFromZero 4
reason = ""           True
reason Is Nothing     True
```

Output — L06.VbSemantics (2/2) captured on the build machine

```
Integer = Nothing      0
"CLASS1" = "class1"    False
AddFee(line.Total)     1000 -> 1035
AddFee((line.Total))   1035 -> 1035
card.LoadingFor(2)     0.25
late bound LoadingFor(2)  0.25
late bound Discount       MissingMemberException
"42" + 1  = 43      "42" & 1  = 421
"CLASS1" = "class1"     True
DateDiff(Year, 31 Dec 2001, 1 Jan 2026)  25
Dim years As Integer = 11 / 12           1
On Error Resume Next   Err.Number 11, ratio 0
```

Every line is a trap you can now name. `"CLASS1" = "class1"` is False in `Program.vb` (Option Compare Binary) and True in `Loose.vb` (Text). The driver born on 31 December 2001 is 24 on 1 January 2026, but `DateDiff` says 25. `On Error Resume Next` swallowed a division by zero and left the result at 0, while `Integer.MaxValue + 1` did not wrap: VB threw `OverflowException`.

4.7 · Four midpoints, three rounding rules



measured: L06.VbSemantics output in 4.6 · C# (int) and Java HALF_UP columns named by the rule they share

No midpoint gets the same answer from all three rules. Banker's rounding matches half-up only at 1.5 and 3.5, and matches truncation only at 0.5 and 2.5, so whichever rule a port reaches for, half of these inputs change. A premium rule that rounds a base, then a loading, then a tax compounds the difference — which is why §7 finds whole-baht mismatches in thousands of quotes.

4.8 · Java instincts vs the VB semantics a faithful port must keep

Java — for comparison the instincts you bring

```java
// What a Java-minded port reaches for, line by line
int base = (int) (sumInsured * rate);    // truncates
long half = Math.round(2.5);             // 3: half up
int years = licenceMonths / 12;          // 18 -> 1

int age = Period.between(dob, start)     // birthdays
                .getYears();

boolean same = code.equals("CLASS1");    // exact case

BigDecimal vat = gross.multiply(VAT_RATE)
    .setScale(0, RoundingMode.HALF_UP);  // half up

int total = net + duty;                  // wraps silently
```

C# samples/L06.ModernRating/VbCompat.cs · #region vb-compat

```csharp
// Legacy VB semantics, named so a reviewer can see them.
public static class VbCompat
{
    // CInt, and implicit Double -> Integer: banker's rounding
    public static int CInt(double value) => checked(
        (int)Math.Round(value, MidpointRounding.ToEven));

    public static int CInt(decimal value) => checked(
        (int)Math.Round(value, MidpointRounding.ToEven));

    // DateDiff(DateInterval.Year, d1, d2): year parts only
    public static int DateDiffYears(
        DateOnly d1, DateOnly d2) => d2.Year - d1.Year;

    // Option Compare Text: current culture, ignoring case,
    // width and kana type
    const CompareOptions Text = CompareOptions.IgnoreCase
        | CompareOptions.IgnoreWidth
        | CompareOptions.IgnoreKanaType;

    public static bool TextEquals(string a, string b) =>
        CultureInfo.CurrentCulture.CompareInfo
            .Compare(a, b, Text) == 0;
}
```

Every Java line is reasonable and every one changes a premium or a guard. The C# port does not sprinkle `MidpointRounding.ToEven` and year subtraction through the rule; it names each legacy semantic once in `VbCompat`, so a reviewer sees exactly which quirks are being preserved — and later, which one is being retired. The `checked(…)` around each cast keeps VB's overflow guard: an out-of-range `CInt` throws in VB, while an unchecked C# cast does not, and Java's `int` arithmetic wraps silently.

5 · VB ↔ C# interop — one runtime, two compilers' rules

Crossing the boundary needs no bridge, but each compiler reads the same metadata its own way. You have met this shape calling Kotlin from Java: a Kotlin top-level function or `object` member is a static method to Java, and `@JvmOverloads` exists because Java ignores Kotlin's default values. Here a VB `Module`'s members are static members to C#, a VB `Optional ByVal` parameter is a C# optional parameter, and `ByRef` is `ref`. In the other direction VB consumes C# records, `decimal?`, `DateOnly` and generics unchanged — the form in §3.4 builds a C# record with named arguments. The VB runtime library, `Microsoft.VisualBasic`, is part of .NET 10 too, so C# code can call `DateAndTime.DateDiff` as easily as VB does.

Case-only names break Visual Basic. Think of a Kotlin API that compiles cleanly but is awkward from Java until you add `@JvmName`: C# happily compiles a class with a `Loading` property and a `loading(int)` method, and VB cannot tell them apart and refuses any use of either (BC31429 in §5.4). The Common Language Specification says identifiers must differ by more than case, and `[assembly: CLSCompliant(true)]` makes the C# compiler warn with CS3005 (Language independence and the CLS). It is the compiler-enforced version of keeping a Kotlin API Java-friendly. If VB code consumes your library — in an estate it usually does — mark the assembly CLS-compliant and let warnings-as-errors gate it.

ByRef writes properties back, and you have no JVM habit for it. Neither Java nor Kotlin can pass a variable by reference, so `ref` and `ByRef` are new to you. VB lets you pass a property to a `ByRef` (or C# `ref`) parameter: it passes a temporary and assigns the result back to the property, so `AddFee(line.Total)` changes `line.Total`. Wrapping the argument in parentheses forces a copy that is discarded (forcing by value). C# does not accept a property as a `ref` argument at all, so a port has to write the copy-back explicitly — or better, return the new value. One more asymmetry: an `Optional ByRef` VB parameter is *required* from C#, which is why the adapter below passes `declineReason` every time.

Sentinels need an adapter. Legacy VB APIs return `-1` for "declined" and report reasons through `ByRef` strings. Wrap them once, in C#, in a method with a modern shape (`decimal?` and an `out` reason), and the rest of the new code never learns the old conventions — the Anti-corruption Layer pattern (Microsoft Learn) at the size of one class.

5.1 · C# calling a VB Module — the adapter at the seam

C# — samples/L06.Parity/LegacyAdapter.cs · #region call-vb

```
// A VB Module's members are static members to C#. An Optional
// ByVal parameter stays optional; Optional ByRef arrives as a
// required ref, so declineReason is always passed.
public static decimal? Quote(QuoteRequest r, out string reason)
{
    string declineReason = "";
    decimal premium = LegacyPremium.CalcPremium(
        r.CoverCode, r.SumInsured,
        r.DateOfBirth.ToDateTime(TimeOnly.MinValue),
        r.StartDate.ToDateTime(TimeOnly.MinValue),
        r.LicenceMonths, r.Claims,
        commercial: r.Commercial, engineCc: r.EngineCc,
        declineReason: ref declineReason);
    reason = declineReason;
    return premium == -1 ? null : premium;   // -1 meant "declined"
}
```

No interop layer, just a call. `LegacyPremium` is a VB `Module`, so C# calls it like a static class. The VB `Optional ByVal commercial` stays optional, but `Optional ByRef declineReason` arrives as a required C# `ref` parameter (leave it out and the compiler reports CS7036), passed here by name. `-1` becomes `null` at this one place.

5.2 · A partner's C# SDK — valid C#, awkward VB

C# — samples/L06.PartnerSdk/PartnerRateCard.cs · #region partner-sdk

```
// A rating partner's C# SDK: legal C#, awkward from Visual Basic.
public class PartnerRateCard
{
    // the canonical loading for one claim (curriculum, "Canonical tariff")
    public decimal Loading { get; set; } = 0.10m;

    // differs from Loading only by case: fine in C#
    public decimal loading(int claims) => claims switch
    {
        0 => 0m,
        1 => Loading,
        2 => 0.25m,
        _ => throw new ArgumentOutOfRangeException(
            nameof(claims), "3+ claims in 5 years are declined"),
    };

    // the case-distinct name a VB caller can actually reach
    public decimal LoadingFor(int claims) => loading(claims);

    // ref + optional: VB may pass a property here
    public static void AddFee(ref decimal amount, decimal fee = 35m) =>
        amount += fee;
}
```

Two members differ only by case. C# compiles this without a warning because the assembly does not claim CLS compliance. `AddFee` takes `ref` plus an optional fee — legal from both languages, with different meanings for a property argument.

5.3 · The VB caller — copy-back through ByRef, and the escape hatch

Visual Basic — samples/L06.VbSemantics/Program.vb · #region byref-property

```
Dim line As New QuoteLine With {.Total = 1000D}
Dim before = line.Total
PartnerRateCard.AddFee(line.Total)          ' property, ByRef
Console.WriteLine($"AddFee(line.Total)     {before} -> {line.Total}")
before = line.Total
PartnerRateCard.AddFee((line.Total))        ' ( ) passes a copy
Console.WriteLine($"AddFee((line.Total))   {before} -> {line.Total}")
```

The parentheses are the whole difference. The first call passes `line.Total` by reference and the property becomes 1035; the second passes a copy and it stays 1035 (captured output in §4.6). `New QuoteLine With {.Total = 1000D}` on the first line is VB's object initializer, the C# `new QuoteLine { Total = 1000m }`; the same keyword also opens a `With line … End With` block. The case-collision call is compiled only with `SHOW_CASE_COLLISION`, so the sample builds; `LoadingFor` is the name a partner should have shipped.

5.4 · Two compile errors worth recognising

Terminal — captured on the build machine · long lines wrapped for print

```
$ dotnet build L06.VbSemantics -p:DefineConstants=SHOW_CASE_COLLISION=True
Program.vb(97,27): error BC31429: 'loading' is ambiguous because multiple kinds
    of members with this name exist in class 'PartnerRateCard'.
Build FAILED.

$ dotnet build L06.VbSemantics -p:DefineConstants=SHOW_MY_COMPUTER=True
Program.vb(107,27): error BC30456: 'Computer' is not a member of 'L06.VbSemantics.My'.
Build FAILED.
```

> **VB reports the lower-case name even though the code says `card.Loading`** — the compiler is case-insensitive, so both members answer to either spelling. The second error is the `My` namespace: a .NET 10 VB console gets no `My.Computer`, so code that leaned on it needs replacing before or during a retarget.

5.5 · Interop rules at the boundary

Feature	C# library, VB caller	VB library, C# caller	Design rule
Static members	`static class` reads like a Module	A `Module`'s members are static	Qualify calls in VB to keep ownership visible
Optional parameters	Optional in VB (`AddFee(x)` omits the fee)	Optional ByVal is optional in C#; `Optional ByRef` is required (CS7036)	Keep optional parameters ByVal if C# will call you
By reference	`ref`/`out` → ByRef; properties copied back	ByRef → `ref`; C# must pass a variable	Return a record or tuple instead
Case-only names	Unusable (BC31429)	VB names are case-insensitive by design	`[assembly: CLSCompliant(true)]` on shared C# libraries
Records, init-only	Consumed; VB 16.9 reads init-only (VB 16.9); `=` compares records by value	VB cannot declare records	Keep shared models in C#
Events	`WithEvents` + `Handles`, or `AddHandler`	`+=` on a VB `Event`	Same CLR event underneath
Sentinels	—	`-1`, empty strings, `ByRef` reasons	One C# adapter converts them to `null` and `out`
String equality	Culture-aware only if you ask	Depends on each file's Option Compare	Name the rule once (`VbCompat.TextEquals`)
VB runtime	Built in	`Microsoft.VisualBasic` ships with .NET 10 — callable from C#	Use it as a test oracle for kept quirks (7.6)

6 · Migration strategy — characterize, strangle, retarget, port

Start with tests that describe what the code does, not what it should do. Characterization tests (golden-master tests) are the opposite of the specification tests behind a 100%-coverage standard: they call the legacy code and pin its current output, quirks included. The VB tests below assert that an 18-month licence counts as two years and that a driver born on 31 December is a year older than one born a day later. They are written in VB, against the VB project, before any port, so they stay valid on both sides of every later change.

Strangle by project, not by file. A VB project calls a C# library with no bridge — lesson 01 (The .NET Platform Map) showed it — so the unit of migration is the project: retarget a VB library to `net10.0`, let C# reference it, then replace it with a C# port behind the same call site. The Strangler Fig pattern's façade (Microsoft Learn) is, inside one process, a project reference plus an adapter; across web estates it is a reverse proxy — lesson 12 (Cloud-Native .NET on AWS & Modernization).

Pick the route by workload. Libraries and console jobs retarget, then are ported when they change. Windows Forms retargets to `net10.0-windows` after the rules move out of handlers — the sample form already calls the C# port. Web Forms needs a new C# UI. VB6 needs a rewrite.

A converter's output is a draft until the grid passes. ICSharpCode CodeConverter converts VB to C# (and back) with Roslyn and says VB → C# quality is much higher than the reverse (icsharpcode/CodeConverter). Microsoft's own tooling is changing: .NET Upgrade Assistant is officially deprecated in favour of the GitHub Copilot modernization agent (.NET Upgrade Assistant overview), and AWS Transform for .NET lists VB.NET as a preview language (AWS Transform docs) — lesson 12 (Cloud-Native .NET on AWS & Modernization) compares those tools and their requirements. Whichever tool writes the C#, the evidence that no premium moved is the same: characterization tests green on the VB, zero mismatches on the grid.

6.1 · Characterization tests in VB — pinning the quirks on purpose

Visual Basic — samples/L06.LegacyRating.Tests/LegacyPremiumCharacterizationTests.vb · #region pin-quirks

```
<Theory>
<InlineData("class1")>
<InlineData("  Class1 ")>
Public Sub CoverCodeIgnoresCaseAndSpaces(code As String)
    Assert.Equal(Quote("CLASS1"), Quote(code))
End Sub

<Fact>
Public Sub AgeCountsYearNumbersNotBirthdays()
    ' Both drivers are 24 on 1 Jan 2026. DateDiff says the one born
    ' on 31 Dec 2001 is 25, so no young-driver loading applies.
    Assert.True(Quote("CLASS1", dob:="2001-12-31") <
                Quote("CLASS1", dob:="2002-01-01"))
End Sub

<Fact>
Public Sub LicenceMonthsRoundToTheNearestEvenYear()
    ' The no-claim discount follows licence years = months / 12,
    ' rounded half to even: 18 -> 2, but 30 -> 2 and 42 -> 4.
    Assert.Equal(Quote("CLASS1", months:=24), Quote("CLASS1", months:=18))
```

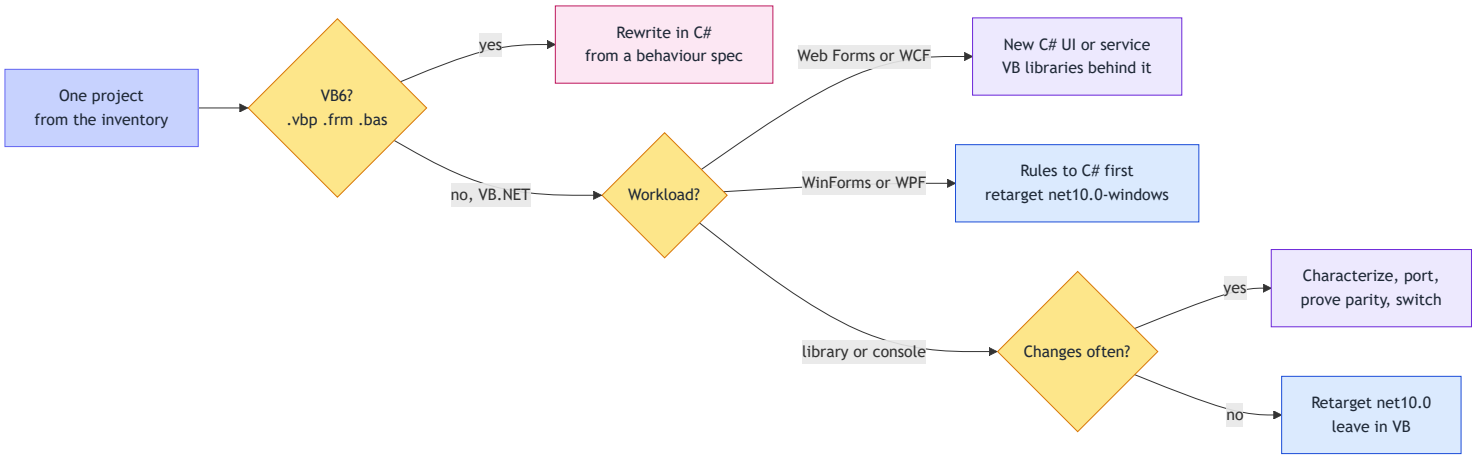
```
    Assert.Equal(Quote("CLASS1", months:=24), Quote("CLASS1", months:=30))
    Assert.Equal(Quote("CLASS1", months:=48), Quote("CLASS1", months:=42))
    Assert.True(Quote("CLASS1", months:=18) < Quote("CLASS1", months:=17))
End Sub

<Fact>
Public Sub LicenceYearsAboveFiveUseTheSixthSlot()
    Assert.Equal(Quote("CLASS1", months:=60), Quote("CLASS1", months:=84))
    Assert.True(Quote("CLASS1", months:=60) < Quote("CLASS1", months:=48))
End Sub

<Fact>
Public Sub AnyClaimLosesTheNoClaimDiscount()
    Assert.Equal(Quote("CLASS1", months:=6, claims:=1),
                 Quote("CLASS1", months:=120, claims:=1))
End Sub
```

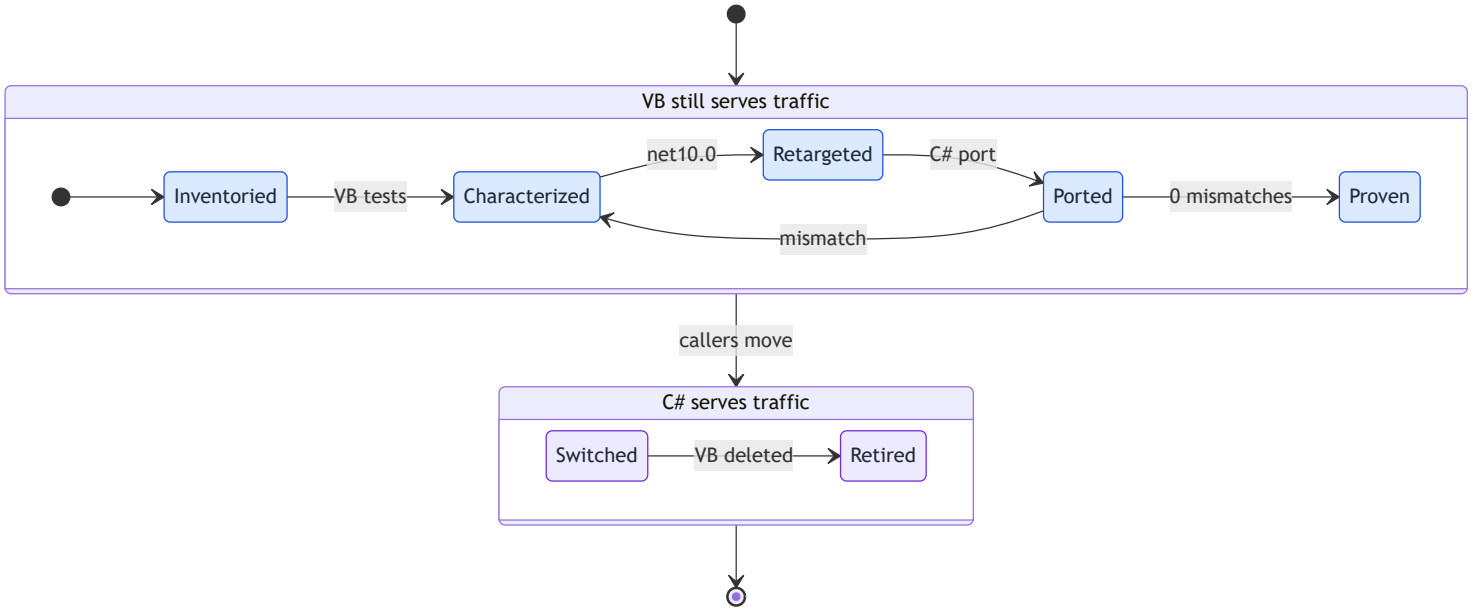
> **These tests would fail code review as specifications, and that is the point.** They assert what production does today: cover codes ignore case and spaces, `DateDiff` ages a 31 December birthday early, 18 months rounds to two years, seven claim-free years reuse the sixth slot, and any claim removes the no-claim discount. xUnit attributes look the same in VB, in angle brackets.

6.2 · Choosing a route for one VB project



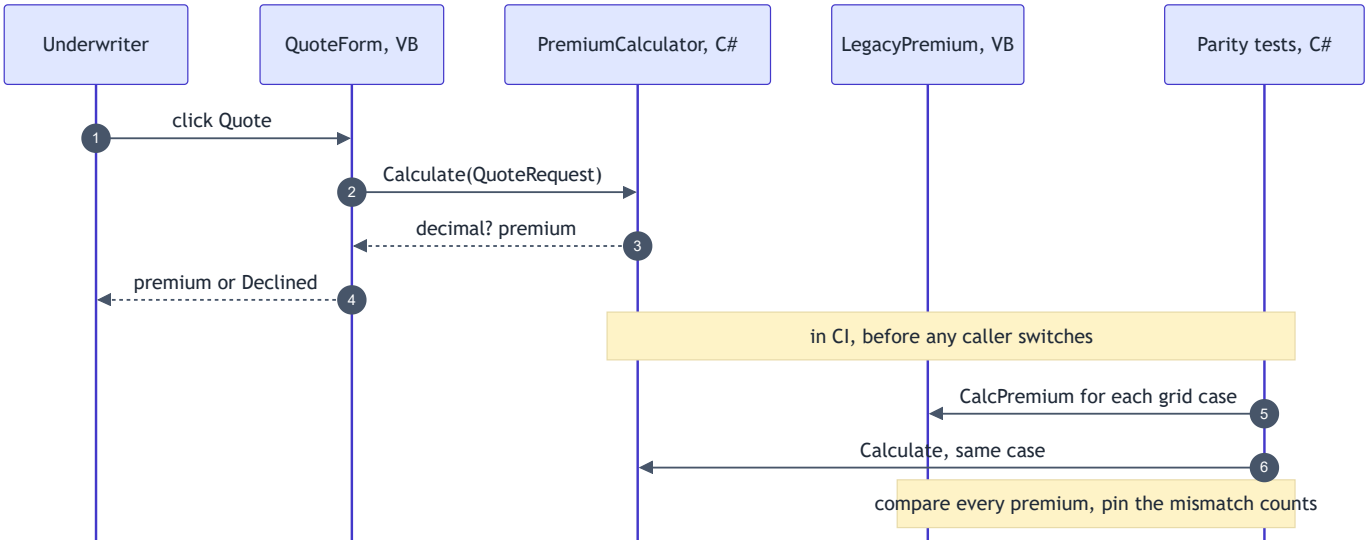
amber = question · blue = stays VB for now · violet = ends in C# · rose = rewrite · every route that ends in C# passes through characterization and parity tests

6.3 · The life of one project in the migration



blue = VB still serves traffic · violet = C# serves traffic · the back-edge is the normal case, not a failure

6.4 · One quote through the strangler — VB screen, C# rule, VB oracle



steps 1-4 = the production path once the screen calls the port · steps 5-6 = the parity run in CI, before callers switch

6.5 · Migration tooling this lesson owns, checked in September 2026

tooling view

7 ICSharpCode CodeConverter tooling · ● VB ↔ C# · ● Verified

Open-source Roslyn converter that uses full project context.

SHIPS AS	Visual Studio extension, `codeconv` dotnet tool, web snippet converter, NuGet library
RELEASE	v10.0.1 on 1 March 2026; since v10.0.0 `codeconv` needs .NET 10, and it still needs Visual Studio 2026 installed
FRAMEWORK	The command line dropped .NET Framework projects in v10; the extension still converts them
GATE	Its output is a draft until the characterization tests and the parity grid pass

8 Characterization and parity tests architecture · ● Your gate · ● Measured

Tests that pin the legacy output and compare the port over a grid of inputs.

COST	324 lines in this lesson; each `dotnet test` run took under a second here
FINDS	All 6 injected port mistakes, each in ≥ 3,072 of 50,176 quotes
WHY	The only tool on this page that knows your business rules

6.6 · Convert with a tool, or rewrite by hand?

Convert mechanically, then prove parity

- Class libraries of pure calculation with characterization tests around them
- Large volumes, where reviewing converted code beats retyping it
- Code whose structure you want to keep recognisable in history and blame
- Always followed by the parity grid — accuracy is measured on your quotes, not assumed

Rewrite by hand, from pinned behaviour

- Windows Forms handlers that mix UI, SQL and business rules — extract the rules first
- Code built on `On Error Resume Next` or late binding
- Web Forms pages: there is no .NET 10 target to convert them to
- Visual Basic 6: COM-era code with no .NET project to convert

7 · Parity testing — prove the port before anyone switches

A parity grid runs old and new code over every combination of the inputs a rule reads, and demands identical outputs. It is the characterization test scaled up: instead of choosing twenty cases, enumerate each dimension — cover code with its casing, sum insured, birth date against start date, licence months, claims, commercial use and engine size — and take the Cartesian product. Here that is 50,176 quote requests, and the console report compares the faithful port and seven one-change variants against the VB original in under a second on the build machine (0.8 s for the whole run, start-up included).

Choose values at the edge of each quirk, not at random. Sums insured that land on .5 after the rate, birth dates on 31 December and 1 January, 6, 18, 30 and 42 months (half a licence year, where the rounding rules part), 60 and 90 months (the sixth table slot), 3,000 and 3,001 cc (the commercial engine-size step), three claims (declined) and codes in mixed case. A grid built that way caught all six injected port mistakes; each changed between 6.1% and 44.9% of premiums. A twenty-case suite picked by hand has to be lucky.

The same grid puts a size on a fix. The team wants decimal rates instead of Double, surely a pure improvement. Injected as its own change it altered 304 of 50,176 premiums (0.6%): `0.35` has no exact binary form, so the first mismatch prices `5,850 × 0.35` as 2,047.4999… in Double (legacy rounds it down to 2,047), while Decimal holds exactly 2,047.5 and rounds half to even up to 2,048. The fix is correct and it still moves prices, so it ships on its own, with that number attached. The birthday rule is larger: a correct age changes 16.1% of quotes, so it is a pricing decision for the business, not a refactoring. Reshaping the port's flat `QuoteRequest`, which mirrors the legacy signature, into the running example's `Vehicle`, `Driver` and `Money` types is another later, separately gated step under this same grid.

Compare results, not implementations. The runner counts "both declined" as a match and an exception as a mismatch, and the xUnit theory pins every count, so a number printed in this lesson fails the build if the samples drift from it.

7.1 · The same loadings in the VB original and the faithful C# port

Visual Basic samples/L06.LegacyRating/LegacyPremium.vb · #region legacy-loadings

```vb
Dim age As Integer = DateDiff(DateInterval.Year,
                              dateOfBirth, startDate)
Dim licenceYears As Integer = licenceMonths / 12

Dim loading As Integer = 0
If age < 25 Then loading += CInt(base * 0.2)
Select Case claims
    Case 0
    Case 1 : loading += CInt(base * 0.1)
    Case 2 : loading += CInt(base * 0.25)
    Case Else
        declineReason = "3+ claims in 5 years"
        Return -1
End Select
If commercial Then
    loading += CInt(base * If(engineCc > 3000, 0.35, 0.25))
End If
```

C# samples/L06.ModernRating/PremiumCalculator.cs · #region port-loadings

```cs
int age = VbCompat.DateDiffYears(
    r.DateOfBirth, r.StartDate);      // calendar years
int licenceYears = VbCompat.CInt(r.LicenceMonths / 12.0);

int loading = 0;
if (age < 25) loading += VbCompat.CInt(@base * 0.2);
switch (r.Claims)
{
    case 0: break;
    case 1: loading += VbCompat.CInt(@base * 0.1); break;
    case 2: loading += VbCompat.CInt(@base * 0.25); break;
    default: return null;             // 3+ claims: declined
}
if (r.Commercial)
    loading += VbCompat.CInt(
        @base * (r.EngineCc > 3000 ? 0.35 : 0.25));
```

Line for line, on purpose. VB's implicit `Integer = Double` conversion becomes an explicit `VbCompat.CInt`; `licenceMonths / 12` becomes `CInt(LicenceMonths / 12.0)`; `DateDiff` becomes `DateDiffYears`. The values are the canonical tariff's: +20% under 25, +10% or +25% for one or two claims, three declined, +25% commercial or +35% above 3,000 cc. The port is uglier than a rewrite would be — that is the price of zero mismatches, paid once and refactored later under the same grid. Every rate is illustrative, not a real tariff.

7.2 · The discount table and the total — VB original and C# port

Visual Basic samples/L06.LegacyRating/LegacyPremium.vb · #region legacy-discount-total

```vb
Dim claimFreeYears As Integer =
    If(claims = 0, licenceYears, 0)
Dim ncb(5) As Decimal    ' six slots: 0 to 5 claim-free years
ncb(1) = 0.2
ncb(2) = 0.25
ncb(3) = 0.3
ncb(4) = 0.4
ncb(5) = 0.5
If claimFreeYears > 5 Then claimFreeYears = 5
Dim discount As Integer =
    (base + loading) * ncb(claimFreeYears)

Dim net As Integer = base + loading - discount
Dim duty As Integer = CInt(net * 0.004)
Dim vat As Decimal = Math.Round((net + duty) * 0.07D)
Return net + duty + vat
```

C# samples/L06.ModernRating/PremiumCalculator.cs · #region port-discount-total

```cs
int claimFree = r.Claims == 0 ? licenceYears : 0;
// six slots, as Dim ncb(5)
decimal[] ncb = [0m, 0.2m, 0.25m, 0.3m, 0.4m, 0.5m];
int discount = VbCompat.CInt(
    (@base + loading) * ncb[Math.Min(claimFree, 5)]);

int net = @base + loading - discount;
int duty = VbCompat.CInt(net * 0.004);
decimal vat = Math.Round(
    (net + duty) * 0.07m, MidpointRounding.ToEven);
return net + duty + vat;
```

`Dim ncb(5)` becomes a six-element array literal, spelled out. The claim-free years are the licence years when there are no claims and 0 otherwise; the `If claimFreeYears > 5` clamp becomes `Math.Min`, and every Decimal or Double converted to Integer (`discount`, `duty`) goes through `VbCompat.CInt` because VB rounds those to even. The VAT line needs no helper: `Math.Round` already rounds to even in both languages.

7.3 · The injected changes — one enum member, one line of the port

C# samples/L06.Parity/NaivePort.cs · #region port-changes

```cs
// One change to the faithful port, injected on its own: six realistic
// translation mistakes, and the decimal fix the team wants to make.
public enum PortChange
{
    None,
    TruncatingCasts,     // CInt(x) written as (int)x
    HalfUpRounding,      // MidpointRounding.AwayFromZero, as Java HALF_UP
    IntegerDivision,     // months / 12 on ints truncates
    BirthdayAge,         // a correct age instead of DateDiff year parts
    CaseSensitiveCodes,  // == instead of Option Compare Text
    FiveElementTable,    // Dim ncb(5) read as five elements
    DecimalRates,        // the planned fix: decimal rates instead of Double
}
```

Six are plausible mistakes and one is a planned improvement. Each member changes exactly one line of the faithful port when `NaivePort` runs it, and each comment names the VB semantic it forgets; `None` is the faithful port. Chart 7.7 counts how many premiums each member moves, and the parity theory in 7.5 pins those counts.

7.4 · The grid — boundary values, multiplied out

C# samples/L06.Parity/QuoteGrid.cs · #region grid

```cs
// Every combination of these values is one parity case.
public static class QuoteGrid
{
    public static readonly string[] CoverCodes =
        ["CLASS1", "Class1", "CLASS2PLUS", "class2plus",
         "CLASS3PLUS", "Class3", "CLASS4"];
    public static readonly decimal[] SumsInsured =
        [205_000m, 287_500m, 312_500m, 450_000m,
         487_500m, 550_000m, 612_500m, 800_000m];
    public static readonly DateOnly[] BirthDates =
        [new(2001, 12, 31), new(2002, 1, 1), new(2002, 6, 15), new(1990, 3, 10)];
    public static readonly DateOnly[] StartDates = [new(2026, 1, 1), new(2026, 7, 1)];
    public static readonly int[] LicenceMonths = [6, 17, 18, 30, 42, 60, 90];
    public static readonly int[] Claims = [0, 1, 2, 3];
```

```
    public static readonly bool[] Commercial = [false, true];
    public static readonly int[] EngineCc = [3_000, 3_001];

    public static IEnumerable<QuoteRequest> All() =>
        from code in CoverCodes
        from sum in SumsInsured
        from dob in BirthDates
        from start in StartDates
        from months in LicenceMonths
        from claims in Claims
        from commercial in Commercial
        from cc in EngineCc
        select new QuoteRequest(code, sum, dob, start, months, claims, commercial, cc);
}
```

7 × 8 × 4 × 2 × 7 × 4 × 2 × 2 = 50,176 cases. Each array holds the values where a quirk changes behaviour: `"Class1"` for Option Compare Text, 31 December 2001 for `DateDiff`, 6/18/30/42 months for half-year rounding, 60 and 90 months for the six-slot table, 3,000 and 3,001 cc for the commercial step, three claims and `CLASS4` so declines are compared too.

7.5 · The parity gate in xUnit

C# — samples/L06.Parity.Tests/GridParityTests.cs · #region parity-tests

```
[Fact]
public void FaithfulPortMatchesLegacyOnEveryGridCase()
{
    var result = ParityRunner.Compare(PremiumCalculator.Calculate);

    Assert.Equal(50_176, result.Cases);
    Assert.True(result.Mismatches == 0, result.FirstMismatch);
}

// The curriculum's worked example: 8,933.71 THB to the satang in the
// canonical tariff; the legacy rule and its port price in whole baht.
[Fact]
public void WorkedExampleIsTheCanonicalTotalInWholeBaht()
{
    var example = new QuoteRequest(
        "CLASS1", 550_000m, new(2002, 6, 15), new(2026, 1, 1), 48, 0);

    Assert.Equal(8_933.71m, CanonicalTariff.Total(example));
    Assert.Equal(8_933m, LegacyAdapter.Quote(example, out _));
    Assert.Equal(8_933m, PremiumCalculator.Calculate(example));
}

// Counts are pinned, so the numbers printed in the lesson stay true.
[Theory]
[InlineData(PortChange.TruncatingCasts, 22_520)]
[InlineData(PortChange.HalfUpRounding, 11_324)]
[InlineData(PortChange.IntegerDivision, 3_072)]
[InlineData(PortChange.BirthdayAge, 8_064)]
[InlineData(PortChange.CaseSensitiveCodes, 16_128)]
[InlineData(PortChange.FiveElementTable, 3_072)]
[InlineData(PortChange.DecimalRates, 304)]
public void GridMeasuresEveryPortChange(PortChange change, int expected)
{
    var result = ParityRunner.Compare(q => NaivePort.Calculate(q, change));

    Assert.Equal(expected, result.Mismatches);
}
```

Two assertions carry the migration. The faithful port must produce zero mismatches over the whole grid. The theory then pins how many premiums each alternative would change — `DecimalRates` at 304, not 0 — so a later refactoring that alters a single quote fails CI. A third test ties the legacy to the canonical tariff: the worked example prices 8,933.71 THB in decimal and 8,933 in whole baht.

7.6 · Checking VbCompat against the Visual Basic runtime

C# — samples/L06.Parity.Tests/GridParityTests.cs · #region vb-runtime

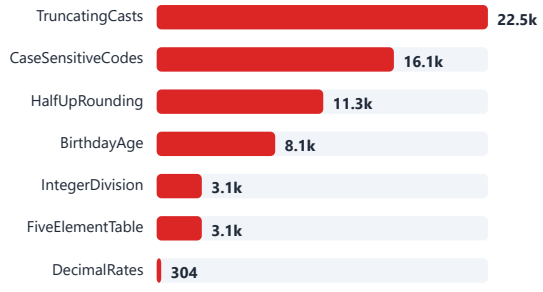
```
// Microsoft.VisualBasic ships with .NET 10, so C# can ask the VB
// runtime itself whether VbCompat reproduces it.
[Fact]
public void VbCompatAgreesWithTheVisualBasicRuntime()
{
    foreach (double v in new[] { 0.5, 1.5, 2.5, 3.5, 3792.5, 17 / 12.0 })
        Assert.Equal(Conversions.ToInteger(v), VbCompat.CInt(v));

    foreach (var born in QuoteGrid.BirthDates)
        foreach (var start in QuoteGrid.StartDates)
            Assert.Equal(
                DateAndTime.DateDiff(DateInterval.Year,
                    born.ToDateTime(TimeOnly.MinValue),
                    start.ToDateTime(TimeOnly.MinValue)),
                VbCompat.DateDiffYears(born, start));
```

```
    foreach (var code in QuoteGrid.CoverCodes)
        Assert.Equal(
            Operators.CompareString(code, "CLASS1", TextCompare: true) == 0,
            VbCompat.TextEquals(code, "CLASS1"));
}
```

The oracle for a quirk is the VB runtime itself, and C# can call it. `Microsoft.VisualBasic` ships with .NET 10, so a C# test can put `Conversions.ToInteger`, `DateAndTime.DateDiff` and `Operators.CompareString(…, TextCompare: true)` beside each `VbCompat` helper. Production code stays on `VbCompat`, whose names say which quirk is kept; the runtime is the test's witness, not a dependency of the port.

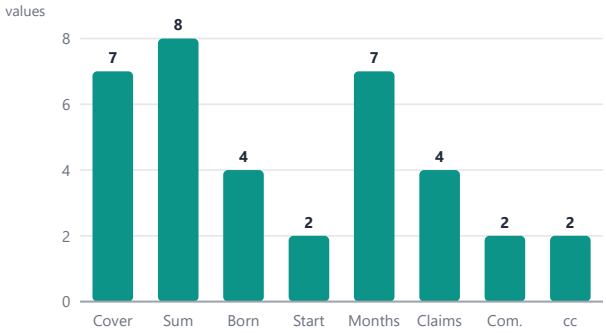
7.7 · Premiums changed by one port change



measured: L06.ParityReport, 50,176 requests per variant

Every mistake is loud on a boundary grid; the planned fix is quiet, not silent. Truncating casts and case-sensitive codes change the most quotes; the quietest mistakes, integer division and a five-element table, still change 3,072 each. `DecimalRates` changed 304 — small enough that a hand-picked suite would miss it, and still real prices.

7.8 · Values per grid dimension



measured: parsed from QuoteGrid.cs at build time; the product is checked against the run

Three dimensions are the widest. Sums insured (8), cover codes (7) and licence months (7) multiply to 392; the other five add a factor of 128. Doubling any one dimension doubles the grid, so add a value only where a rule changes behaviour — the run stays under a second.

8 · Hands-on — run the estate in miniature

Nine projects model one migration from end to end. `L06.LegacyRating` is the VB rule and `L06.LegacyRating.Tests` pins it. `L06.ModernRating` is the faithful C# port, `L06.Parity` holds the grid, the adapter and the injected variants, `L06.Parity.Tests` gates them, and `L06.ParityReport` prints the numbers used in §7. `L06.PartnerSdk` and `L06.VbSemantics` demonstrate the traps, and `L06.WinFormsVb` is the VB screen that already calls C#.

Work in the order a real migration does. Run the VB characterization tests, then the parity tests, then the report. Only when all three agree with the numbers in this lesson is the estate in a known state.

Then break it on purpose, twice, and read what fails. Make `VbCompat.CInt(double)` a plain `(int)` cast: on the build machine 7 of the 11 parity tests failed, including the VB-runtime check in 7.6, and the faithful-port test named its first case — `CLASS1 205000 … 6m 0cl com 3000cc: legacy 5781, port 5780`. Put the cast back, then change `Option Compare Text` to `Binary` at the top of `LegacyPremium.vb`: 3 of the 11 VB characterization tests failed, every one of them a lower- or mixed-case cover code. That is the division of labour you want — the VB tests guard the legacy behaviour, the grid guards the port — and neither needed a debugger.

8.1 · Commands

Terminal

```
# build, test and run every lesson-06 sample (the .NET 10 SDK must be on PATH)
python 0-script/verify_samples.py --only 6

# the pieces one at a time
cd lesson-06-vbnet-legacy-estates/samples
dotnet test L06.LegacyRating.Tests       # VB characterization tests
dotnet test L06.Parity.Tests             # C# parity gate
dotnet run --project L06.ParityReport    # the numbers in section 7
dotnet run --project L06.VbSemantics     # the traps in section 4

# reproduce the two compile errors from section 5.4
dotnet build L06.VbSemantics -p:DefineConstants=SHOW_CASE_COLLISION=True
dotnet build L06.VbSemantics -p:DefineConstants=SHOW_MY_COMPUTER=True
```

Run the VB tests before the C# ones. The characterization suite is the specification the parity gate relies on; if it fails, the legacy rule changed and every number in §7 is suspect. The two `-p:DefineConstants` builds are meant to fail — they reproduce §5.4.

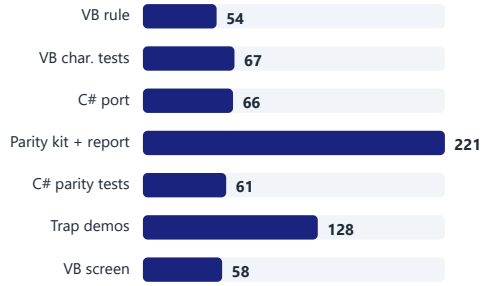
8.2 · What you should see

Output — L06.ParityReport captured on the build machine

```
Parity grid        50,176 quote requests
Faithful C# port   0 mismatches
Port variants, one change at a time:
  TruncatingCasts     22,520   44.9%
  HalfUpRounding      11,324   22.6%
  IntegerDivision      3,072    6.1%
  BirthdayAge          8,064   16.1%
  CaseSensitiveCodes  16,128   32.1%
  FiveElementTable     3,072    6.1%
  DecimalRates           304    0.6%
Worked example  CLASS1 550000 born 2002-06-15 start 2026-01-01 48m 0cl
  VB legacy 8933  C# port 8933  canonical 8933.71
Sample  class1 287500 born 2001-12-31 start 2026-01-01 18m 0cl
  VB legacy 4864  C# port 4864
First HalfUpRounding mismatch
  CLASS1 205000 born 2001-12-31 start 2026-01-01 6m 0cl: legacy 4625, port 3700
First DecimalRates mismatch
  CLASS2PLUS 487500 born 2001-12-31 start 2026-01-01 6m 0cl com 3001cc: legacy 8484, port 8485
```

Two lines are worth checking by hand (canonical illustrative tariff). The worked example is 550,000 × 2.1% = 11,550, +20% for age 23 = 13,860, −40% for four claim-free years = net 8,316.00, stamp duty 33.26, VAT 584.45, total **8,933.71** in decimal; the legacy prices the same quote at 8,933 because it rounds every step to whole baht. The `class1` sample: 287,500 × 2.1% = 6,037.5, which banker's rounding keeps as 6,038; 18 months counts as 2 licence years (1.5 rounds to even), so the discount is 25% of 6,038 = 1,509.5, stored as 1,510; net 4,528, duty 18, VAT 318, total 4,864 — the mixed-case code costs nothing, and the faithful port agrees to the baht. In the first half-up mismatch the licence is 6 months, half a year: banker's rounding stores 0 years and no discount (4,625), half-up stores 1 year and a 20% discount (3,700).

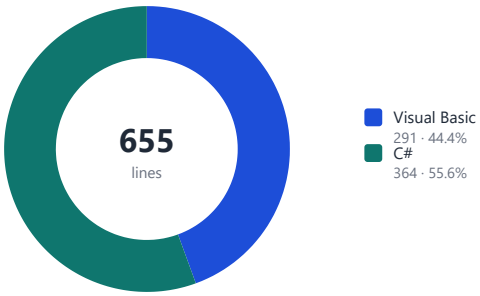
8.3 · Lines of code by migration role



non-blank, non-comment lines, grouped by role · measured at build time

The port is not the big part. The parity kit and the two test projects together are larger than the VB rule and its C# port combined — the realistic ratio when the goal is zero changed premiums.

8.4 · Sample code by language



measured: projects with .vb files counted as Visual Basic

A migration repository is bilingual by design. VB is still 44% of the sample code: the rule, its tests, the trap demos and the screen stay VB until parity is proven and callers move. 11 VB test cases and 11 C# test cases run side by side in one build.

⚠ Traps — what looks right to a Java, TypeScript or Python engineer

1. **`(int)x` for `CInt(x)`.** CInt rounds half to even; the cast truncates. On the grid this one change moved 44.9% of premiums.
2. **`months / 12` on integers.** VB's `/` returns Double and the Integer assignment rounds; C# integer division truncates — 6.1% of premiums.
3. **Trusting the project's Option Strict.** A file-level `Option Strict Off` or `Option Compare Text` overrides it for that file.
4. **Porting `Dim ncb(5) as new decimal[5]`.** It compiles, passes every test below five claim-free years, and throws for 6.1% of the grid.
5. **Fixing while porting.** A correct age calculation changed 16.1% of premiums, and even the "obvious" Decimal-for-Double swap changed 0.6%. Port the quirk, pin it, and change it in a separate, signed-off release.

✓ Checkpoint — you are ready for lesson 07 when

1. `python 0-script/verify_samples.py --only 6` reports 9/9 passed, including 11 VB characterization cases and 11 C# parity cases.
2. You can read `LegacyPremium.vb` top to bottom (panels 4.1, 7.1 and 7.2) and point at the line each of the six port mistakes would change (7.3).
3. You can explain why `AddFee(line.Total)` and `AddFee((line.Total))` behave differently, and write the C# equivalent of each.
4. Given a VB project from an inventory, you can place it on diagram 6.2 and defend the route.

🔍 Check yourself

1. What does `CInt(2.5)` return, what does C# `(int)2.5` return, and which `MidpointRounding` value makes `Math.Round` match VB?
2. What does `Dim years As Integer = 18 / 12` store, and which two VB rules produce that value?
3. Why can VB not call a C# class with both `Loading` and `loading()`, and which attribute makes C# warn?
4. Which VB workloads have templates in the .NET 10 SDK, and how do you put a VB rating library behind a web API?
5. What separates a characterization test from a parity grid, and why did the decimal-rates change belong in its own release?
6. What does Microsoft mean by a consumption-only strategy for Visual Basic?
7. `LegacyPremium.vb` starts with `Option Compare Text`: what does its `Select Case` do with `" class1 "`, and which helper reproduces that in the C# port?

Lesson 06 in one line: VB.NET reads like C# in English keywords, but Option Strict Off, round-half-to-even conversions, `/`, `DateDiff`, `Dim a(n)` and Option Compare Text change numbers — so characterize the VB, port it faithfully, and let a parity grid decide when callers switch.
Next: lesson 07 (Solutions, MSBuild & Mono-repos) — how solutions, MSBuild and mono-repos hold C# and VB projects in one build.